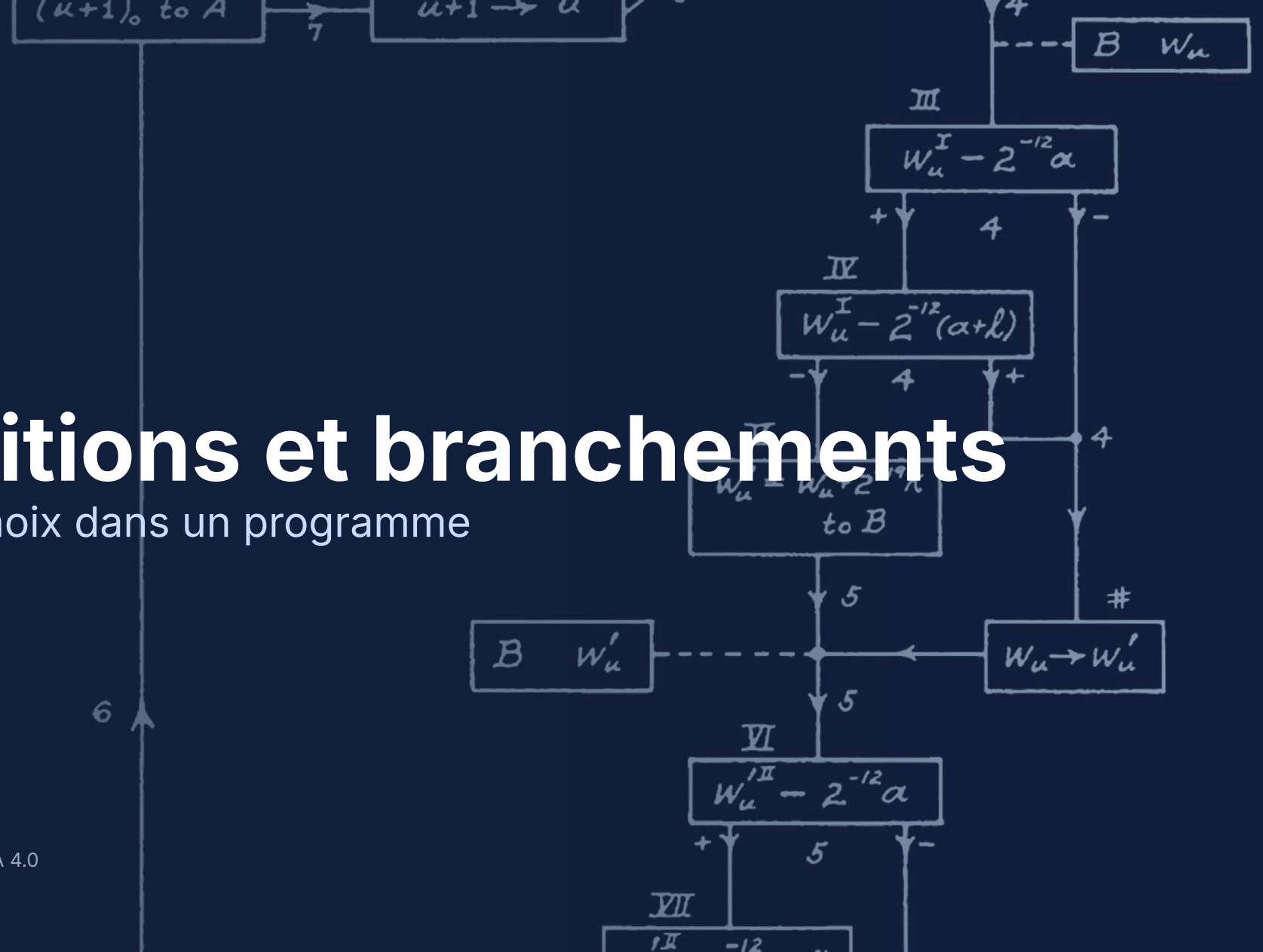
Conditions et branchements

Faire des choix dans un programme



Nicholas Wolff · CC BY-SA 4.0

Sommaire

1

Conditions

Comparaisons entre nombres

Comparaisons entre chaînes de caractères

Opérateurs de comparaison en Python

2

Branchements conditionnels

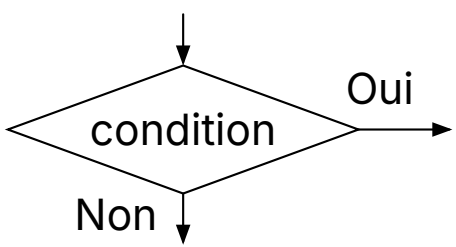
`if …`

`if … else …`

Inverser une condition

Un branchement dans un branchement

Condition d'un branchement conditionnel



Dans un branchement conditionnel, on écrit une **condition**.

Une condition est une question dont la réponse est **oui** (`True`) ou **non** (`False`).

Dans une condition, il y a toujours au moins une variable présente. Par exemple : `x == 0`

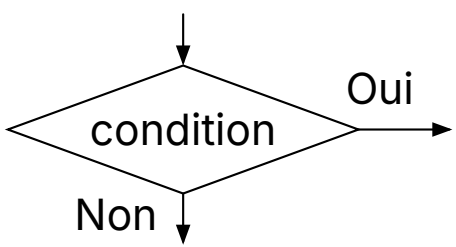
Si la valeur de x est effectivement égale à `0`, alors la condition s'évalue en **oui**, et on prendra la branche notée **Oui**. Sinon, on prendra la branche notée **Non**.

On peut aussi écrire d'autres comparaisons, par exemple : `delta > 0`

Pour « supérieur ou égal », on écrit >= (qui représente ≥), par exemple : `delta >= 0`

Pour « différent de », on écrit `!=` (qui représente ≠), par exemple : `delta != 0`

Condition avec des chaînes de caractères



On peut aussi écrire des conditions avec des chaînes de caractères.

Comparaisons

- On différencie les lettres majuscules, minuscules, et les caractères accentués.
- Les chaînes de caractères sont ordonnées par ordre alphabétique.

Exemples :

```
"Rouge" == "Rouge"   →  True
"rouge" == "Rouge"   →  False
"A" < "B"            →  True
"y" > "z"            →  False
"1" > 1              →  Erreur
```

Attention : ne pas confondre comparaison et assignation

Il ne faut pas confondre une comparaison qui teste l'égalité entre deux choses avec une assignation qui change la valeur d'une variable.

Assignation	=
Comparaison	==

Prenons un exemple pour contraster les deux :

```
x = 10          # Assigne la valeur 10 à x
print(x == 0)   # Affiche False car x vaut 10 et 10 n'est pas égal à 0
print(x)        # Affiche 10 car x n'a pas été modifié par la comparaison précédente
x = 0
print(x)        # Affiche 0 car à la ligne précédente on a assigné 0 à x
```

Opérateurs de comparaison en Python

En français	En maths	En Python
Supérieur	>	>
Inférieur	<	<
Supérieur ou égal	≥	>=
Inférieur ou égal	≤	<=
Égal	=	== (deux signes =)
Différent	≠	!=

Sommaire

1

Conditions

Comparaisons entre nombres

Comparaisons entre chaînes de caractères

Opérateurs de comparaison en Python

2

Branchements conditionnels

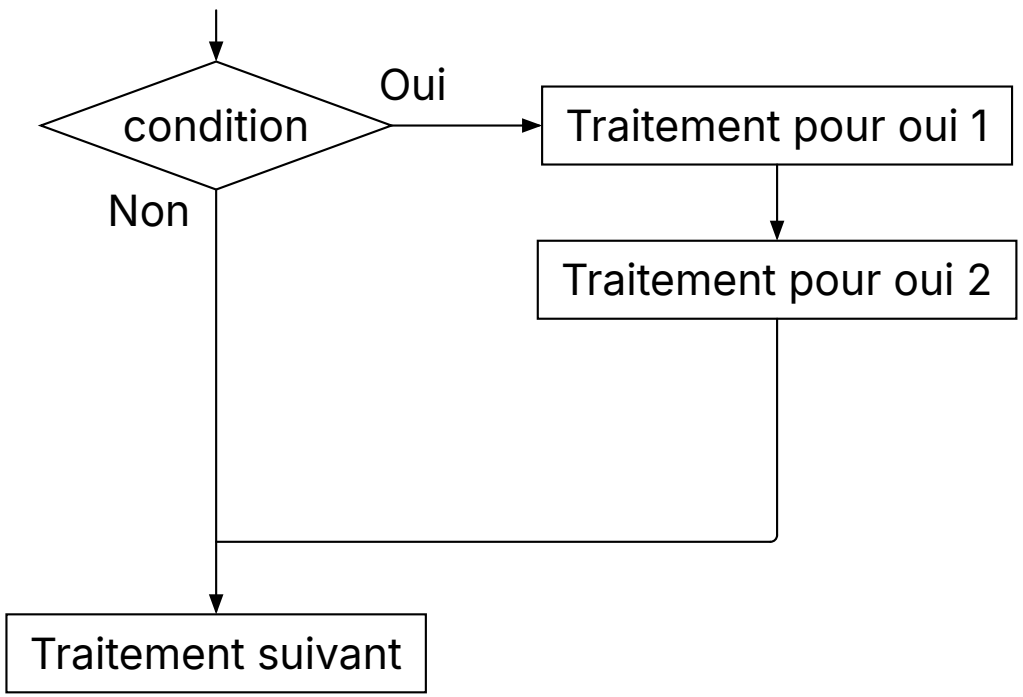
`if …`

`if … else …`

Inverser une condition

Un branchement dans un branchement

Branchement conditionnel sans alternative



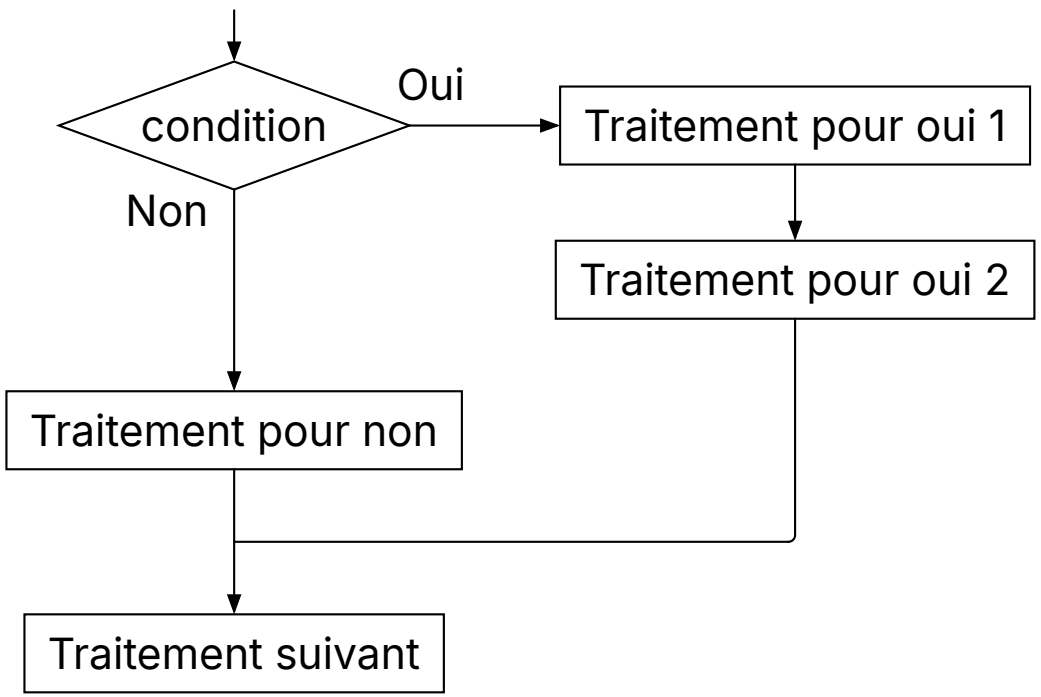
Algorigramme

```
if condition:
    traitement_pour_oui_1
    traitement_pour_oui_2


traitement_suivant
```

Programme

Branchement conditionnel avec alternative



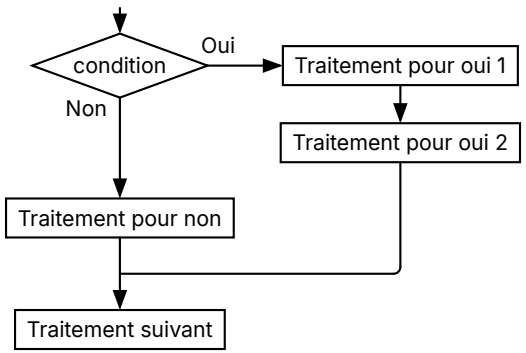
Algorigramme

```
if condition:
    traitement_pour_oui_1
    traitement_pour_oui_2
else:
    traitement_pour_non
traitement_suivant
```

Programme

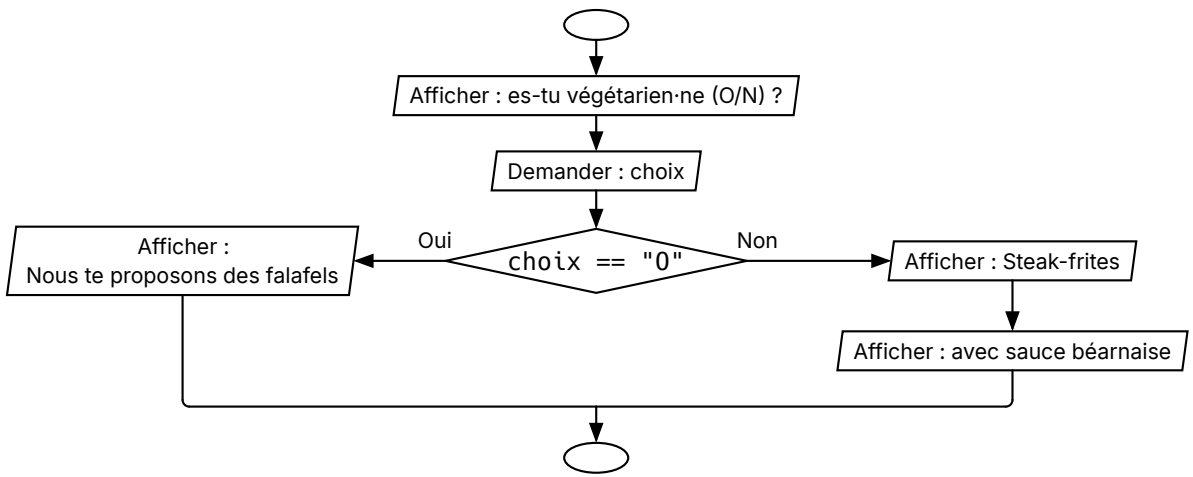
Exemple de programme concret avec `if` et `else`

Forme générale (pour illustrer l'idée de if else)




```
if condition:
    traitement_pour_oui_1
    traitement_pour_oui_2
else:
    traitement_pour_non
traitement_suivant
```

Un algorigramme **concret**



Le programme **concret** correspondant

```
1 choix = input("es-tu végétarien·ne (O/N) ? ")
2 if choix == "O":
3     print("Nous te proposons des falafels")
4 else:
5     print("Steak-frites")
6     print("avec sauce béarnaise")
```

Exercice : accorder en nombre

Voici un programme qu'on a écrit précédemment et qui fonctionne :

```
1 nom = input("quel est ton nom ? ")
2 âge = int(input("quel est ton âge ? "))
3 print("Bonjour", nom, "tu as", âge, "ans.")
```

Quand on exécute ce programme, on voit apparaître un petit problème de grammaire :

```
quel est ton nom ? Bob
quel est ton âge ? 1
Bonjour Bob tu as 1 ans.
```

Exercice · 6 min

Modifier ce programme pour qu'il accorde correctement :

- quand l'âge est 0 ou 1, il faut écrire « an » ;
- à partir de deux ans, il faut écrire « ans ».

Exercice : accorder en nombre

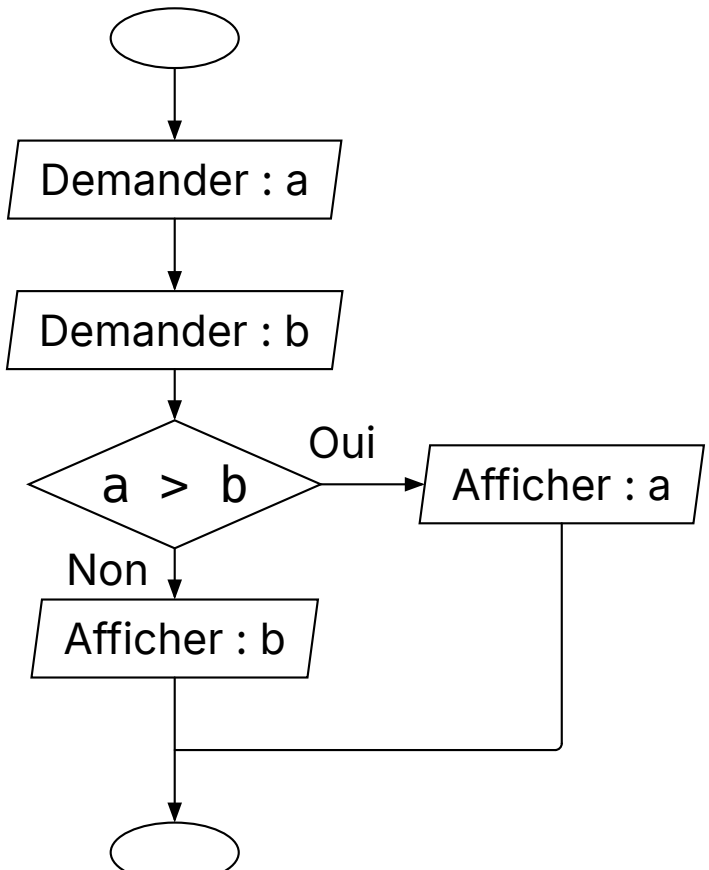
Modifier ce programme pour qu'il accorde correctement :

- quand l'âge est 0 ou 1, il faut écrire « an » ;
- à partir de deux ans, il faut écrire « ans ».

```
# Première manière de faire
nom = input("quel est ton nom ? ")
âge = int(input("quel est ton âge ? "))
if âge > 1:
    print("Bonjour", nom, "tu as", âge, "ans.")
else:
    print("Bonjour", nom, "tu as", âge, "an.")
```

```
# Deuxième manière de faire,
# pour ne pas répéter tout le print
nom = input("quel est ton nom ? ")
âge = int(input("quel est ton âge ? "))
if âge > 1:
    suffixe = "ans."
else:
    suffixe = "an."
print("Bonjour", nom, "tu as", âge, suffixe)
```

Exercice : maximum



Algorigramme

Exercice · 5 min

Écrire un programme Python qui réalise l'algorigramme de gauche. L'algorithme affiche le plus grand de deux nombres demandés.

Programme

Exercice : maximum

Écrire un programme Python qui réalise l'algorigramme de la diapo précédente. L'algorithme affiche le plus grand de deux nombres demandés.

```
a = int(input("1er nombre : "))
b = int(input("2e nombre : "))
if a > b:
    print("Le plus grand nombre est :", a)
else:
    print("Le plus grand nombre est :", b)
```

En regardant ce programme, on pourrait avoir l'impression qu'il ne fonctionne pas bien quand a et b sont égaux (parce que, quand ils sont égaux, on passe dans le `else` et on annonce que b est le plus grand).

Tester le programme. Revenir soit :

- avec un code qui répare le problème ;
- avec une justification de pourquoi le programme est correct.

Exercice : inverser une condition

Un programme correct en Python :

```python
âge = int(input("Ton âge : "))
if âge < 18:
    print("mineur·e")
else:
    print("majeur·e")
```

Exercice · 2 min

Écrire la condition manquante pour que ce second programme fasse exactement la même chose que le premier (noter qu'on a inversé majeur et mineur) :

```python
âge = int(input("Ton âge : "))
if _____________:
    print("majeur·e")
else:
    print("mineur·e")
```

Exercice : inverser une condition

Un programme correct en Python :

```python
âge = int(input("Ton âge : "))
if âge < 18:
    print("mineur·e")
else:
    print("majeur·e")
```

Exercice

Écrire la condition manquante pour que ce second programme fasse exactement la même chose que le premier (noter qu'on a inversé majeur et mineur) :

```python
âge = int(input("Ton âge : "))
if not (âge < 18):
    print("majeur·e")
else:
    print("mineur·e")
```

Ou, en réécrivant la comparaison : `âge >= 18`.

Exercice : inverser des conditions

Exercice · 2 min

Inverser les conditions suivantes avec `not`.

Condition	Avec `not`
`a = 1`	
`b == 1`	
`x < 0`	
`y <= x`	

Exercice : inverser des conditions

Exercice · 2 min

Inverser les conditions suivantes avec `not`.

Puis les inverser sans `not`.

Condition	Avec `not`	Sans `not`
`a = 1`	`# C'est une assignation !`	
`b == 1`	`not (b == 1)`	
`x < 0`	`not (x < 0)`	
`y <= x`	`not (y <= x)`	

Exercice : inverser des conditions

Exercice

Inverser les conditions suivantes avec `not`.

Puis les inverser sans `not`.

Condition	Avec not	Sans not
`a = 1`	`# C'est une assignation !`	
`b == 1`	`not (b == 1)`	`b != 1`
`x < 0`	`not (x < 0)`	`x >= 0`
`y <= x`	`not (y <= x)`	`y > x`

Forme générale des branchements conditionnels

On retrouve cette manière d'expliquer dans l'aide-mémoire Python.

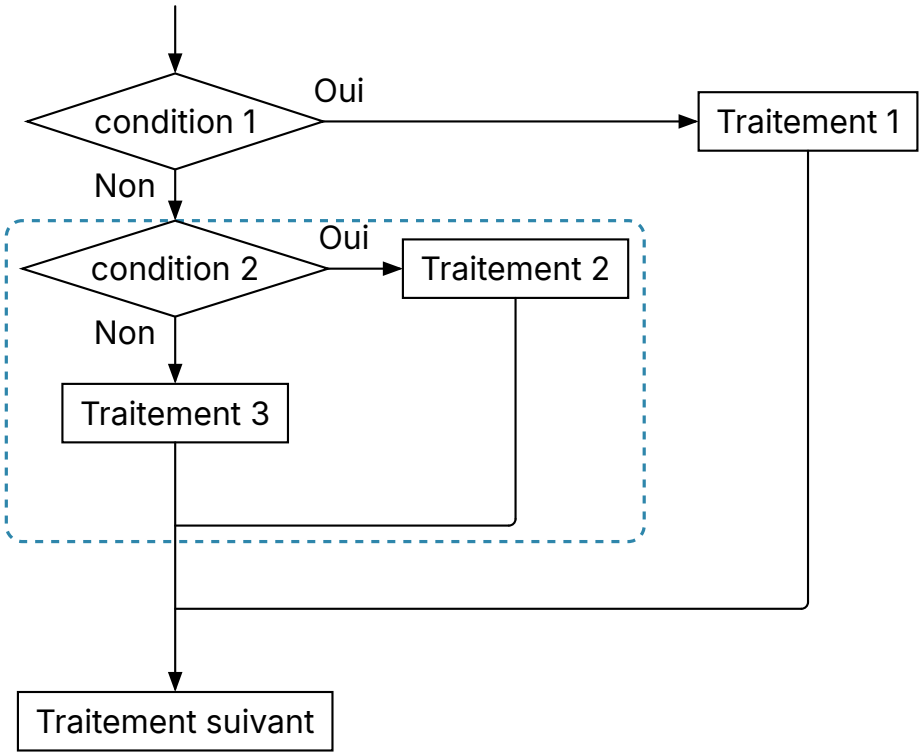
```
if condition:
    instructions si condition vraie
else:    # Optionnel
    instructions si condition fausse
instructions inconditionnelles
```

Important :

- Après une *condition* et après `else`, on écrit deux-points (`:`).
- Juste ensuite, il faut un bloc indenté, c.-à-d. décalé vers la droite.
  Par chance, Thonny indente automatiquement la ligne qui suit les `:`.
- Pour désindenter (décaler moins), il faut appuyer sur la touche retour-arrière ⟵.

Un branchement dans un branchement

Une branche peut contenir un autre branchement (encadré) : avec deux conditions, on obtient trois traitements possibles.




```
if condition_1:
    traitement_1
else:
    if condition_2:
        traitement_2
    else:
        traitement_3
traitement_suivant
```

Plusieurs niveaux d'indentation


```
0   1   2
if condition_1:
    traitement_1
else:
    if condition_2:
        traitement_2
    else:
        traitement_3
traitement_suivant
```

Après le `:`, un bloc indenté d'un niveau de plus : niveau **1**.

Le `else` du premier `if` : de retour au niveau **0**, celui de son `if`.

Le deuxième `if` est dans le bloc du `else` : niveau **1**.

Après son `:`, un niveau de plus : niveau **2**.

Le `else` du deuxième `if` : niveau **1**, celui de son `if`.

De retour au niveau **0** : exécuté dans tous les cas.

Sources

Images

Herman Goldstine et John von Neumann – un diagramme de flux, *Planning and Coding of Problems for an Electronic Computing Instrument* (1947, domaine public)
commons.wikimedia.org